

Cây link/cut

Nguồn gốc: *Link/cut tree*

english/Wikipedia_link-cut_tree.pdf

Tổng quan

Cây link/cut là một cấu trúc dữ liệu dùng để biểu diễn một rừng, tức một tập các cây có gốc. Cấu trúc này hỗ trợ các thao tác sau:

- thêm vào rừng một cây chỉ gồm một đỉnh;
- cho một đỉnh trong một cây, tách đỉnh đó cùng toàn bộ cây con của nó khỏi cây hiện tại;
- gắn một đỉnh làm con của một đỉnh khác;
- cho một đỉnh, tìm gốc của cây chứa nó. Khi thực hiện thao tác này trên hai đỉnh khác nhau, ta có thể kiểm tra chúng có thuộc cùng một cây hay không.

Rừng được biểu diễn có thể gồm các cây rất sâu. Nếu chỉ lưu rừng bằng các cây con trở cha thông thường, việc tìm gốc của một đỉnh có thể mất nhiều thời gian. Ngược lại, nếu mỗi cây trong rừng được biểu diễn bằng cây link/cut, ta có thể xác định một phần tử thuộc cây nào trong thời gian khấu hao $O(\log n)$. Hơn nữa, ta có thể nhanh chóng cập nhật tập các cây link/cut khi rừng biểu diễn thay đổi; cụ thể, việc hợp nhất bằng thao tác link và tách bằng thao tác cut đều có thời gian khấu hao $O(\log n)$.

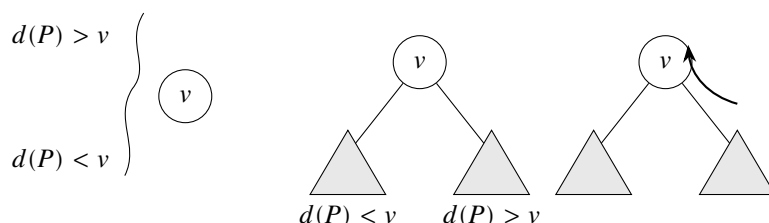
Thao tác	Trung bình	Trường hợp xấu nhất
Link	$O(\log n)$ khấu hao	$O(\log n)$
Cut	$O(\log n)$ khấu hao	$O(\log n)$
Path	$O(\log n)$ khấu hao	$O(\log n)$
FindRoot	$O(\log n)$ khấu hao	$O(\log n)$

Cây link/cut chia mỗi cây trong rừng được biểu diễn thành các đường đi rời nhau theo đỉnh. Mỗi đường đi được biểu diễn bằng một cấu trúc dữ liệu phụ trợ, thường là cây splay; bài báo gốc ra đời trước cây splay nên dùng cây tìm kiếm nhị phân thiên lệch. Các đỉnh trong cấu trúc phụ trợ được sắp theo độ sâu của chúng trong cây được biểu diễn tương ứng. Trong biến thể *phân hoạch ngẫu nhiên*, các đường đi được xác định bởi các đường đi và đỉnh được truy cập gần nhất, tương tự cây Tango. Trong *phân hoạch theo kích thước*, đường đi được xác định theo con nặng nhất, tức con có nhiều hậu duệ nhất, của đỉnh đang xét. Cách này làm cấu trúc phức tạp hơn, nhưng giảm chi phí thao tác từ $O(\log n)$ khấu hao xuống $O(\log n)$ trong trường hợp xấu nhất. Cấu trúc có ứng dụng trong nhiều bài toán luồng mạng và trên các tập dữ liệu động.

Trong công trình gốc, Sleator và Tarjan gọi cây link/cut là “cây động”, hoặc “dynamic dyno trees”.

1 Cấu trúc

Ta xét một cây mà mỗi đỉnh có số con tùy ý và các con không có thứ tự, rồi tách cây đó thành các đường đi. Cây ban đầu này được gọi là **cây được biểu diễn**. Các đường đi được biểu diễn bên trong bằng các cây phụ trợ; ở đây ta dùng cây splay. Trong cây phụ trợ, thứ tự từ trái sang phải biểu diễn đường đi từ gốc đến đỉnh cuối của đường đi đó. Những đỉnh kề nhau trong cây được biểu diễn nhưng không nằm trên cùng một đường đi ưu tiên, và do đó không nằm trong cùng một cây phụ trợ, được nối bằng một **con trở cha của đường**. Con trở này được lưu ở gốc của cây phụ trợ biểu diễn đường đi.



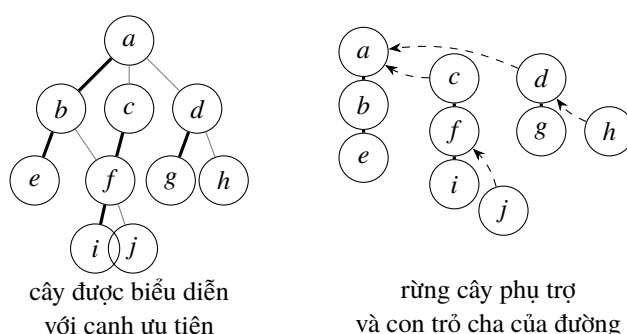
Hình 1: Các đỉnh trong cây phụ trợ được sắp theo độ sâu: phần bên trái của v nông hơn, còn phần bên phải sâu hơn.

1.1 Đường đi ưu tiên

Khi thực hiện truy cập đến một đỉnh v trong cây được biểu diễn, đường đi được đi qua trở thành **đường đi ưu tiên**. **Con ưu tiên** của một đỉnh là đứa con cuối cùng nằm trên đường truy cập, hoặc là rỗng nếu lần truy cập cuối cùng là đến chính v , hoặc nếu chưa từng truy cập nhánh cây cụ thể đó. **Cạnh ưu tiên** là cạnh nối con ưu tiên với đỉnh đang xét.

Trong một phiên bản khác, các đường đi ưu tiên được xác định bởi con nặng nhất.

2 Thao tác



Hình 2: Một cây link/cut biến các đường đi ưu tiên thành một rừng cây phụ trợ; các cạnh ngoài đường đi được lưu bằng con trở cha của đường.

Các thao tác ta quan tâm là

`FindRoot(Node v)`, `Cut(Node v)`, `Link(Node v, Node w)`, `Path(Node v)`.

Mọi thao tác đều được cài đặt bằng thủ tục con `Access(Node v)`. Khi ta truy cập một đỉnh v , đường đi ưu tiên của cây được biểu diễn được đổi thành đường đi từ gốc R của cây được biểu diễn đến đỉnh v . Nếu

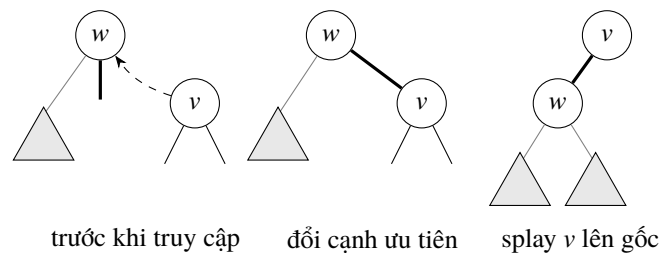
một đỉnh trên đường truy cập trước đó có con ưu tiên u , còn đường đi mới đi qua con w , thì cạnh ưu tiên cũ bị xóa và được đổi thành con trở cha của đường, còn đường đi mới sẽ đi qua w .

2.1 Access

Sau khi thực hiện Access tại đỉnh v , đỉnh này không còn con ưu tiên nào và nằm ở cuối đường đi. Vì các đỉnh trong cây phụ trợ được khóa theo độ sâu, mọi đỉnh ở bên phải v trong cây phụ trợ phải bị tách ra. Với cây splay, thao tác này khá đơn giản: ta splay tại v , đưa v lên gốc của cây phụ trợ. Sau đó ta tách cây con phải của v ; đó chính là mọi đỉnh nằm dưới v trên đường đi ưu tiên cũ. Gốc của cây bị tách sẽ có một con trở cha của đường, và ta cho con trở đó trở đến v .

Tiếp theo ta đi ngược lên cây được biểu diễn đến gốc R , phá và đặt lại đường đi ưu tiên khi cần. Để làm điều này, ta đi theo con trở cha của đường từ v ; vì v hiện là gốc của cây phụ trợ, ta truy cập trực tiếp được con trở đó. Nếu đường đi chứa v đã chứa gốc R thì, do các đỉnh được khóa theo độ sâu, R là đỉnh trái nhất trong cây phụ trợ; khi đó con trở cha của đường là rỗng và thao tác truy cập kết thúc.

Ngược lại, con trở dẫn đến một đỉnh w trên một đường đi khác. Ta muốn phá đường đi ưu tiên cũ của w rồi nối nó với đường đi chứa v . Để làm vậy, ta splay tại w và tách cây con phải của nó, đặt con trở cha của đường của cây bị tách thành w . Vì mọi đỉnh được khóa theo độ sâu, và mọi đỉnh trên đường đi chứa v đều sâu hơn mọi đỉnh trên đường đi chứa w , ta chỉ cần nối cây của v làm con phải của w . Sau đó ta splay lại tại v ; do v là con của gốc w , thao tác này chỉ xoay v lên gốc. Ta lặp toàn bộ quá trình cho đến khi con trở cha của đường của v là rỗng; khi đó v nằm trên cùng đường đi ưu tiên với gốc R của cây được biểu diễn.



Hình 3: Trong Access, các đường đi ưu tiên cũ bị phá thành con trở cha của đường, còn đỉnh đang truy cập được splay lên gốc cây phụ trợ.

2.2 FindRoot

FindRoot là thao tác tìm gốc của cây được biểu diễn chứa đỉnh v . Vì thủ tục Access đưa v vào đường đi ưu tiên, trước hết ta thực hiện Access. Khi đó v nằm trên cùng đường đi ưu tiên, và do đó trong cùng cây phụ trợ, với gốc R . Vì các cây phụ trợ được khóa theo độ sâu, gốc R là đỉnh trái nhất của cây phụ trợ. Ta chỉ cần liên tục đi sang con trái của v cho đến khi không thể đi tiếp; đỉnh thu được là gốc R . Gốc này có thể nằm ở độ sâu tuyến tính, là trường hợp xấu nhất đối với cây splay, nên ta splay nó để lần truy cập sau nhanh hơn.

2.3 Cut

Ở đây ta muốn cắt cây được biểu diễn tại đỉnh v . Trước hết ta truy cập v . Việc này đặt mọi phần tử nằm thấp hơn v trong cây được biểu diễn vào cây con phải của v trong cây phụ trợ. Mọi phần tử hiện nằm trong cây con trái của v là các đỉnh cao hơn v trong cây được biểu diễn. Do đó ta tách con trái của v ; cây này vẫn giữ liên kết với cây được biểu diễn ban đầu thông qua con trở cha của đường. Lúc này v là gốc

của một cây được biểu diễn. Việc truy cập v cũng phá đường đi ưu tiên bên dưới v , nhưng cây con đó vẫn giữ liên kết với v thông qua con trở cha của đường.

2.4 Link

Nếu v là gốc của một cây và w là một đỉnh trong cây khác, thao tác Link nối hai cây chứa v và w bằng cách thêm cạnh (v, w) , làm cho w trở thành cha của v . Để thực hiện, ta truy cập cả v và w trong hai cây tương ứng rồi đặt w làm con trái của v . Vì v là gốc, còn các đỉnh trong cây phụ trợ được khóa theo độ sâu, sau khi truy cập v thì v không có con trái trong cây phụ trợ, vì nó có độ sâu nhỏ nhất. Thêm w làm con trái trên thực tế làm cho w trở thành cha của v trong cây được biểu diễn.

2.5 Path

Trong thao tác này, ta muốn tính một hàm gộp nào đó trên tất cả các đỉnh hoặc cạnh thuộc đường đi từ gốc R đến đỉnh v , chẳng hạn sum , min , max , increase , v.v. Ta thực hiện Access tại v , qua đó nhận được một cây phụ trợ chứa toàn bộ các đỉnh trên đường đi từ gốc R đến v . Cấu trúc dữ liệu có thể được mở rộng để lưu thông tin cần truy vấn, chẳng hạn giá trị nhỏ nhất, giá trị lớn nhất, hoặc tổng chi phí trong cây con; thông tin trên một đường đi khi đó có thể được trả về trong thời gian hằng số sau khi đã thực hiện truy cập.

3 Phân tích

Cut và Link có chi phí $O(1)$ cộng với chi phí của Access. FindRoot có cận trên khẩu hao $O(\log n)$ cộng với chi phí của Access. Tùy cách cài đặt, cấu trúc dữ liệu có thể được mở rộng bằng các thông tin bổ sung, chẳng hạn đỉnh có giá trị nhỏ nhất hoặc lớn nhất trong các cây con, hoặc tổng. Do đó Path có thể trả về thông tin này trong thời gian hằng số cộng với cận của Access.

Vì vậy, để tìm thời gian chạy, phần còn lại là chặn chi phí của Access. Thao tác Access dùng splay, mà ta biết có cận trên khẩu hao $O(\log n)$. Phần phân tích còn lại xét số lần cần splay. Số này bằng số lần thay đổi con ưu tiên, tức số cạnh thay đổi trong đường đi ưu tiên, khi ta đi ngược lên cây.

Ta chặn Access bằng kỹ thuật **phân rẽ nặng-nhẹ**.

3.1 Phân rẽ nặng-nhẹ

Kỹ thuật này gọi một cạnh là nặng hoặc nhẹ tùy theo số đỉnh trong cây con. Ký hiệu $\text{size}(v)$ là số đỉnh trong cây con của v trong cây được biểu diễn. Một cạnh được gọi là **nặng** nếu

$$\text{size}(v) > \frac{1}{2} \text{size}(\text{parent}(v)).$$

Do đó mỗi đỉnh có nhiều nhất một cạnh nặng. Cạnh không nặng được gọi là **nhẹ**.

Độ sâu nhẹ là số cạnh nhẹ trên đường đi từ gốc đến một đỉnh v . Ta có

$$\text{độ sâu nhẹ} \leq \lg n,$$

vì mỗi lần đi qua một cạnh nhẹ, số đỉnh giảm ít nhất một nửa so với cây con của cha.

Vậy một cạnh trong cây được biểu diễn có thể thuộc một trong bốn loại: nặng-ưu tiên, nặng-không ưu tiên, nhẹ-ưu tiên, hoặc nhẹ-không ưu tiên. Trước hết ta chứng minh cận trên $O(\log^2 n)$.

3.1.1 Cận trên $O(\log^2 n)$

Mỗi thao tác splay trong Access đóng góp hệ số $\log n$, nên để chứng minh cận trên $O(\log^2 n)$, ta cần chặn số lần truy cập bởi $\log n$.

Mỗi lần cạnh ưu tiên thay đổi, một cạnh ưu tiên mới được tạo ra. Vì vậy ta đếm số cạnh ưu tiên được tạo. Trên một đường đi bất kỳ có nhiều nhất $\log n$ cạnh nhẹ, nên có nhiều nhất $\log n$ cạnh nhẹ chuyển thành ưu tiên.

Số cạnh nặng chuyển thành ưu tiên có thể là $O(n)$ trong một thao tác cụ thể, nhưng chỉ là $O(\log n)$ theo nghĩa khấu hao. Trên một dãy thao tác, ban đầu có thể có $n - 1$ cạnh nặng chuyển thành ưu tiên, vì toàn bộ cây được biểu diễn có nhiều nhất $n - 1$ cạnh nặng. Từ đó trở đi, số cạnh nặng chuyển thành ưu tiên bằng số cạnh nặng đã chuyển thành không ưu tiên ở một bước trước. Với mỗi cạnh nặng chuyển thành không ưu tiên, phải có một cạnh nhẹ chuyển thành ưu tiên. Ta đã thấy số cạnh nhẹ có thể chuyển thành ưu tiên nhiều nhất là $\log n$. Vì vậy trong m thao tác, số cạnh nặng chuyển thành ưu tiên là

$$m \log n + (n - 1).$$

Với đủ nhiều thao tác, cụ thể $m > n - 1$, giá trị trung bình là $O(\log n)$.

3.1.2 Cải thiện thành cận trên $O(\log n)$

Ta đã chặn số lần thay đổi con ưu tiên bởi $O(\log n)$. Nếu chứng minh được mỗi lần thay đổi con ưu tiên có chi phí khấu hao $O(1)$, ta sẽ chặn được thao tác Access bởi $O(\log n)$. Điều này được thực hiện bằng phương pháp thế năng.

Gọi $s(v)$ là số đỉnh nằm dưới v trong cây của các cây phụ trợ. Khi đó hàm thế năng là

$$\Phi = \sum_v \log s(v).$$

Ta biết chi phí khấu hao của thao tác splay được chặn bởi

$$\text{cost}(\text{splay}(v)) \leq 3(\log s(\text{root}(v)) - \log s(v)) + 1.$$

Sau khi splay, v là con của đỉnh w được trở bởi con trở cha của đường, nên ta có

$$s(v) \leq s(w).$$

Sử dụng bất đẳng thức này cùng chi phí khấu hao của truy cập, ta thu được một tổng dạng lồng nhau được chặn bởi

$$3(\log s(R) - \log s(v)) + O(\text{số lần thay đổi con ưu tiên}),$$

trong đó R là gốc của cây được biểu diễn. Vì số lần thay đổi con ưu tiên là $O(\log n)$ và $s(R) = n$, ta thu được cận khấu hao $O(\log n)$.

4 Ứng dụng

Cây link/cut có thể được dùng để giải bài toán kết nối động trên đồ thị không chu trình. Với hai đỉnh x và y , chúng liên thông khi và chỉ khi

$$\text{FindRoot}(x) = \text{FindRoot}(y).$$

Một cấu trúc dữ liệu khác cũng dùng được cho mục đích này là cây Euler tour.

Trong bài toán luồng cực đại, cây link/cut có thể cải thiện thời gian chạy của thuật toán Dinic từ $O(V^2E)$ xuống $O(VE \log V)$.

5 Xem thêm

- Cây splay.
- Phương pháp thể năng.

6 Đọc thêm

- Sleator, D. D.; Tarjan, R. E. (1983). “A Data Structure for Dynamic Trees”. *Proceedings of the thirteenth annual ACM symposium on Theory of computing – STOC ’81*, p. 114. doi:<https://doi.org/10.1145/800076.802464>.
- Sleator, D. D.; Tarjan, R. E. (1985). “Self-Adjusting Binary Search Trees”. *Journal of the ACM*, 32(3): 652. doi:<https://doi.org/10.1145/3828.3835>.
- Goldberg, A. V.; Tarjan, R. E. (1989). “Finding minimum-cost circulations by canceling negative cycles”. *Journal of the ACM*, 36(4): 873. doi:<https://doi.org/10.1145/76359.76368>. Ứng dụng cho tuần hoàn chi phí nhỏ nhất.
- “Link-Cut trees”, ghi chú bài giảng về cấu trúc dữ liệu nâng cao, Spring 2012, bài giảng 19. Giáo sư Erik Demaine; người ghi chép: Justin Holmgren (2012), Jing Jian (2012), Maksim Stepanenko (2012), Mashhood Ishaque (2007).
- <http://compgeom.cs.uiuc.edu/~jeffe/teaching/datastructures/2006/notes/07-linkcut.pdf>